

Dynamic Network Slicing

(Redes Definidas por Software)

Ano Letivo 2025/2026



Tiago Diogo
PG60308



Gonçalo Soares
PG60262



Cláudia Faria
PG60240

Grupo 02

June 4, 2026

Dynamic Network Slicing (*Redes Definidas por Software*)

Tiago Diogo - PG60308, Gonalo Soares - PG60262, Cludia Faria - PG60240
(Instituio): Universidade do Minho, Braga

Abstract—Este trabalho apresenta a conceo e implementao de uma arquitetura de Network Slicing dinmica sobre uma infraestrutura SDN programvel em P4. A topologia consiste em trs switches P4 interligando seis hosts distribudos por duas sub-redes, onde o router central (*d2*) concentra a lgica de classificao de slices, controlo de largura de banda por medidores, e uma firewall stateful baseada em Bloom Filters. O isolamento de trfego  garantido atravs de metadados de pipeline, evitando o overhead de cabealhos fsicos adicionais. Para lidar com trfego excedente, o sistema recorre a containers Linux (*cNFs*) com filas HTB para Traffic Shaping dinmico, ativados autonomamente por telemetria gRPC. A resilincia da rede  assegurada por um mecanismo de Self-Healing capaz de detetar e recuperar falhas no plano de dados em menos de 5 segundos, sem interveno humana. Os resultados obtidos validam o isolamento efetivo entre slices, o cumprimento dos SLAs de dbito e a robustez da arquitetura perante falhas induzidas.

Index Terms—Software-Defined Networking (SDN), P4, Network Slicing, Network Function Virtualization (NFV), Traffic Shaping, Self-Healing, Telemetria.

I. INTRODUO

A. Cenrios

A necessidade crescente de infraestruturas de rede flexveis e dinmicas, capazes de servir mltiplos utilizadores ou servios (*tenants*) com requisitos de desempenho e isolamento distintos, motivou o desenvolvimento deste projeto. Em vez da implementao fsica de mltiplas redes isoladas, uma abordagem dispendiosa e pouco escalvel, a soluo adotada baseia-se na partilha de uma infraestrutura fsica comum, particionada em *Network Slices* (slices de rede).

O cenrio prtico implementado  constitudo por uma topologia em linha composta por trs switches programveis em P4 (*d1*, *d2* e *d3*), interligando um total de seis hosts (*h1* a *h6*) distribudos por duas sub-redes distintas. No ncleo desta infraestrutura, o dispositivo *d2* atua estritamente como um router L3, centralizando a lgica de encaminhamento inter-subredes, a firewall stateful e a aplicao de *meters* de controlo de largura de banda. Em contrapartida, os restantes dispositivos perifricos (*d1* e *d3*) operam como switches L2 tradicionais com aprendizagem dinmica de endereos MAC (*self-learning*). Esta segregao de funes permite demonstrar de forma clara o isolamento de recursos por slice e o impacto das polticas de QoS no trfego que atravessa o router central.

B. Conceitos

Este projeto fundamenta-se nos princpios de SDN (Software-Defined Networking), onde o plano de controlo

(*Control Plane*)  completamente desacoplado do plano de dados (*Data Plane*). Os conceitos-chave explorados incluem:

- **Network Slicing:** Tcnica de virtualizao que permite segmentar a infraestrutura fsica em redes lgicas independentes, garantindo slices com atribuio isolada de dbito, prioridades e polticas de segurana.
- **Programabilidade (P4):** Criao de pipelines customizados no plano de dados do router *d2*, permitindo a inspeo profunda de pacotes, classificao em slices e validao de fluxos de segurana via *Bloom Filters*  velocidade da linha (*line-rate*).
- **Service Chaining e cNFs:** Encaminhamento direcionado de fluxos excedentes para funes de rede virtualizadas em containers (*cNFs*), estendendo a capacidade do plano de dados com lgicas complexas de filas de espera (*Traffic Shaping* via HTB/Linux) que excedem a expressividade nativa do P4.

C. Relevancia

A relevncia desta arquitetura reside na sua capacidade de resilincia e adaptao autonoma face a picos de carga ou congestionamento. Os perfis de trfego numa rede real so inerentemente volteis. O sistema desenvolvido responde a este desafio atravs de um controlador inteligente que recolhe continuamente mtricas de telemetria (dbito de entrada e sada) por gRPC. Quando o trfego gerado por uma slice ultrapassa o limite contratualizado (SLA), o plano de controlo ativa de forma inteiramente dinmica os mecanismos de amortecimento no plano de dados estendido (*cNF*), protegendo a largura de banda das restantes slices sem necessidade de interveno humano.

II. DESIGN DO PROTOCOLO

O design do protocolo de *Network Slicing* desenvolvido nesta arquitetura prescinde da insero de um cabealho fsico adicional (*shim header*) no plano de dados. Em vez disso, implementou-se uma estratgia de **encapsulamento lgico baseada em metadados de pipeline**. Suportada pela flexibilidade da linguagem P4, esta abordagem otimiza a eficincia de processamento, eliminando o overhead associado a operaes contnuas de insero e remoo de cabealhos a cada salto.

A. Slice Header Format e Field Semantics

A identificao da slice de rede  qual um pacote pertence  gerida de forma intrnseca atravs do campo *slice_id*, alojado na estrutura de metadados do *switch* (struct

metadata). O formato e a semântica deste identificador lógico processam-se da seguinte forma:

- **Formato do Identificador:** O campo `meta.slice_id` possui uma largura de 8 bits (`bit<8>`), permitindo a identificação teórica de até 256 slices distintas.
- **Classificação (Ingress Gateway):** No bloco de entrada (`MyIngress`), a tabela `slice_classifier` avalia o endereço IP de origem do pacote (`hdr.ipv4.srcAddr`) efetuando uma correspondência exata (*exact match*). A ação `set_slice` atribui imediatamente o respetivo `slice_id` ao contexto do pacote.
- **Enforcement (Medição):** Uma vez classificado, o pacote é sujeito ao medidor `sliceMeter`, que utiliza o `slice_id` como chave (*index*) para aplicar as políticas rigorosas de *Traffic Shaping* e largura de banda definidas pelo controlador.
- **QoS e Escalonamento (Egress Gateway):** No bloco de saída (`MyEgress`), a semântica do `slice_id` dita o tratamento de escalonamento (*scheduling*). A slice 3 (Gold) recebe prioridade máxima (`std_meta.priority = 1`), a slice 2 (Silver) recebe prioridade média, e a slice 1 (Bronze) prioridade mínima. Ao sair para a camada física no `MyDeparser`, o pacote retoma o seu formato original, garantindo compatibilidade universal.

B. Justificação

A decisão arquitetural de utilizar metadados persistentes de *pipeline* em detrimento de um protocolo de encapsulamento físico (com *EtherType* dedicado) baseia-se em três pilares fundamentais de engenharia de redes:

- 1) **Desempenho Line-Rate:** A manipulação de cabeçalhos físicos a cada salto exigiria o *parsing* e *deparsing* de campos customizados, bem como recalcular constantemente os *checksums* do IPv4 e TCP/UDP. O uso de metadados internos permite que o *switch* aplique as regras de isolamento à velocidade máxima da porta (*line-rate*), mitigando a latência de processamento no *Data Plane*.
- 2) **Eliminação de Overhead e MTU:** A introdução de cabeçalhos físicos aumenta o tamanho global do *frame*. Numa rede de alto débito, isto pode causar problemas de fragmentação por incompatibilidade de MTU (*Maximum Transmission Unit*). A marcação via metadados garante transparência na rede.
- 3) **Isolamento Lógico Eficaz:** No paradigma SDN programável, o objetivo principal do *Slicing* é o isolamento de recursos e rotas. O `meta.slice_id` acompanha o ciclo de vida do pacote durante toda a sua travessia pelo *switch*, ativando os *meters*, *counters* e filas prioritárias, cumprindo funcionalmente todos os requisitos de QoS exigidos sem a penalização de desempenho dos protocolos de túnel tradicionais.

III. DISTRIBUIÇÃO DE FUNÇÕES (FUNCTION PLACEMENT)

A arquitetura deste projeto foi desenhada respeitando o princípio fundamental das redes SDN: a separação estrita entre as decisões de encaminhamento (*Control Plane*) e a execução rápida dessas decisões (*Data Plane*). Para cada função de rede (*Network Function*), o seu local de execução foi escolhido com base nas necessidades de latência, retenção de estado e expressividade de código.

A. Plano de Dados (Hardware P4)

O plano de dados corre diretamente no *pipeline* dos *switches* P4, operando pacote a pacote à velocidade da linha (*line-rate*). As funções aqui colocadas exigem execução em poucos segundos, por vezes milissegundos:

- **Encaminhamento e Slicing:** A classificação de slices (`slice_classifier`) e o encaminhamento L2/L3 são executados em *hardware*. O P4 utiliza medidores (`sliceMeter`) para somar o tráfego e detetar fluxos que excedam a largura de banda contratualizada, aplicando de imediato regras de prioridade estritas (*scheduling*) nas filas de saída.
- **Stateful Firewall:** Integrada no *router* central, atua em duas frentes sem consultar o controlador: descarta pacotes sem cabeçalho TCP/UDP válido (bloqueando efetivamente pedidos ICMP/Pings) e utiliza *Bloom Filters* em memória física para garantir que apenas o tráfego originado na rede interna (LAN) tem permissão de passagem, bloqueando ligações não solicitadas da WAN.

B. Plano de Dados Estendido (cNF de Traffic Shaping)

A funcionalidade de *Traffic Shaping* exige retenção temporal de pacotes e a implementação de filas complexas, lógicas que ultrapassam a expressividade semântica do P4 (sem memória dinâmica para *buffers*).

- **Gestão de Filas (Scheduler HTB):** O abrandamento de tráfego foi delegado para um container Linux (cNF), atuando como uma extensão fluida do plano de dados. Este serviço recebe os pacotes marcados em excesso pelo P4, armazena-os em filas geridas pelo protocolo *Hierarchical Token Bucket* (`tc/HTB`) e devolve-os à rede a uma taxa rigorosamente controlada para mitigar drops absolutos (*drops*).

C. Plano de Controlo (Control Plane)

O plano de controlo atua como o "cérebro" da rede. Não processa o tráfego pesado dos utilizadores de forma a não causar estrangulamentos, operando de forma puramente reativa e de gestão.

- **Plugins (Controlador SDN):** A inteligência de orquestração foi distribuída por quatro módulos especializados:
 - `slicing_app.py`: Responsável por injetar as políticas de *Slicing* e Rotas L3. Contem a lógica de *Self-Healing*, reagindo a eventos de reinício do

equipamento para repor o *pipeline* sem intervenção manual.

- `slicing_counting.py`: O motor de telemetria. Consome via gRPC os contadores de *bytes/drops*, calculando as larguras de banda em tempo real. É este *plugin* que toma a decisão de enviar os sinais SIGUSR1/2 para ativar ou desativar a cNF de *Traffic Shaping* perante congestionamentos.
- `self_learning.py`: Processa os pacotes com *EtherType* interno 0x1234 clonados para o CPU, aprendendo dinamicamente os endereços MAC e instalando rotas *unicast* nos *switches* de borda.
- `table_surveyor.py`: Efetua sondagens (*polling*) a cada 10 segundos ao *hardware*, emitindo alertas críticos (STATE_RESET, ENTRY_REMOVED) caso detete alterações indevidas nas regras de encaminhamento.
- **cNFs de Sinalização (ARP e DHCP)**: Serviços como o ARP Proxy e o DHCP Server correm em containers isolados de infraestrutura, atuando especificamente na gestão da sub-rede que aloja os *hosts* h4, h5 e h6 (LAN 2). Delegar estes protocolos de diálogo longo (*handshakes* e atribuição de *leases* dinâmicos) para cNFs dedicadas liberta ciclos de CPU vitais do controlador SDN central e evita a ocupação desnecessária de memória nas tabelas de alta velocidade do *switch* P4.

IV. TOPOLOGIA

A. Desenho da Rede e Papéis dos Dispositivos

A topologia de rede foi modelada no ambiente Mininet recorrendo a uma arquitetura linear bem definida, estruturada para isolar funções de comutação periférica e encaminhamento de núcleo. A infraestrutura é composta por três *switches* programáveis em P4 e seis *hosts*:

- **Dispositivos de Borda (d1 e d3)**: Atuam estritamente como *switches* de acesso Camada 2 (L2). O dispositivo d1 agrega os *hosts* da LAN 1, enquanto o d3 agrega os *hosts* da LAN 2. Estes equipamentos não possuem tabelas de encaminhamento estático, operando de forma autónoma através do *plugin* de *self-learning* para comutação local baseada em endereços MAC.
- **Dispositivo Central (d2)**: Atua como o *Router* L3 e *Gateway* central da rede. Concentra toda a lógica crítica de segurança e controlo de tráfego, nomeadamente o *Network Slicing* (tabelas de classificação e medidores), a Firewall *Stateful* (Bloom Filters) e o encaminhamento entre sub-redes (LPM). Possui ainda a porta 5 (TS_PORT) explicitamente dedicada a interligar a cNF de *Traffic Shaping* ao caminho de dados.

B. Endereçamento e Interligações

O plano de endereçamento divide o sistema em duas sub-redes distintas (/24), interligadas através do núcleo L3 com parâmetros de rede que simulam ligações de *backbone* estáveis

(largura de banda de 50 Mbps, atraso de 10ms e 1% de perda programada nos canais d1-d2 e d3-d2):

- **LAN 1 (Sub-rede 10.0.1.0/24)**: Aloja os *hosts* emissores h1, h2 e h3, configurados com IPs estáticos e com rota padrão apontada para o *Gateway* 10.0.1.254 no *router* d2. Cada um destes *hosts* é a origem de uma *slice* de rede dedicada.
- **LAN 2 (Sub-rede 10.0.2.0/24)**: Aloja os *hosts* recetores h4, h5 e h6. O endereçamento nesta rede é inteiramente dinâmico; no arranque, os *hosts* invocam o comando `dhclient`, delegando a atribuição de IPs, rota de *Gateway* (10.0.2.254) e DNS para a cNF DHCP Server ligada à porta 4 do *router*.

C. Mapeamento de Slices (Slice-to-Path Mapping)

O mapeamento de recursos é efetuado dinamicamente no *router* d2 através da tabela `slice_classifier`, que associa o IP de origem do pacote a uma das três *slices* lógicas com limites de débito independentes:

- **Slice 1 (Bronze)**: Associada ao h1 (10.0.1.1), com um débito contratualizado (SLA) de 10 Mbps (`rate: 1250000 Bytes/s`).
- **Slice 2 (Silver)**: Associada ao h2 (10.0.1.2), com um débito alocado de 15 Mbps (`rate: 1875000 Bytes/s`).
- **Slice 3 (Gold)**: Associada ao h3 (10.0.1.3), com prioridade máxima de processamento e débito alocado de 25 Mbps (`rate: 3125000 Bytes/s`).

O encaminhamento dos fluxos segue caminhos lógicos diferenciados baseados no estado de carga da rede: em condições normais (*under-limit*), o tráfego utiliza o *Fast-Path* de comutação por hardware do P4, onde o campo `std_meta.priority` do *switch* dita a precedência de saída (prioridade 1 para Gold, prioridade 2 para Silver e prioridade 3 para Bronze). Caso uma *slice* entre em incumprimento (*over-limit*), o medidor (`sliceMeter`) deteta o excesso e altera dinamicamente o caminho lógico do tráfego, desviando os pacotes não conformes para fora do *Fast-Path* em direção à porta 5 (TS_PORT) para amortecimento na cNF.

V. IMPLEMENTAÇÃO

A execução do projeto assentou numa arquitetura modular baseada em *plugins* Python, comunicando com o plano de dados através do protocolo gRPC/P4Runtime. Esta estrutura de código permitiu isolar as diferentes lógicas de orquestração (*Slicing*, Telemetria, Aprendizagem e Resiliência) em componentes independentes. Destacam-se as seguintes decisões técnicas e respetiva implementação:

A. Estrutura de Código (Code Structure)

A lógica do sistema foi dividida de acordo com a sua responsabilidade:

- **Plano de Dados (P4)**: O ficheiro `d2_slicing.p4` centraliza a topologia L3, englobando tabelas de classificação (`slice_classifier`), *Meters* de controlo de banda e *Bloom Filters* para a *Stateful Firewall*. Em complemento,

o ficheiro `l2switch.p4` corre nos dispositivos de borda (d1 e d3), executando a comutação L2 e clonando tráfego desconhecido para o controlador.

- **Plano de Controlo (Python):** O controlador SDN carrega os *plugins* baseados na classe estendida `PluginBase`, que operam em *threads* concorrentes e comunicam entre si através de um barramento de eventos (ex: `PACKET_IN`, `ENTRY_REMOVED`).
- **Plano de Dados Estendido e Sinalização (cNFs):** A execução de serviços avançados de rede foi isolada em *containers* Linux, configurados através de ficheiros `YAML` e controlados por pequenos *scripts* Python que atuam diretamente no sistema operativo do container:

- `traffic_shaping.py` e `traffic_shaping.yaml`: Implementam o *scheduler* HTB (*tc* do Linux) para a gestão inteligente das filas de espera e abrandamento dinâmico de tráfego das *slices*.
- `dhcp_server.py` / `.yaml` e `arp_proxy.py` / `.yaml`: Processam a atribuição dinâmica de IPs (*leases*) e a resolução de endereços L2/L3, correndo como serviços autónomos (em portas `*veth*`) para libertar o controlador principal de operações de *handshake* prolongadas.

B. Telemetria e Ativação Dinâmica (Dynamic Traffic Shaping)

A gestão de congestionamento foi desenhada para ser autónoma e inteligente, não penalizando os recursos da cNF de *Traffic Shaping* de forma desnecessária. O *plugin* `slicing_counting.py` atua como o motor analítico da rede. Num ciclo de *polling* a cada 5 segundos, efetua a leitura dos registos `sliceCounter` (total que tenta passar) e `passedCounter` (total que passa após o medidor).

O diferencial (*delta*) entre estes dois contadores permite identificar inequivocamente *drops* induzidos pelo excesso de tráfego. Caso se registem perdas ativas de pacotes em qualquer *slice* (`delta_drop_pkts > 0`), o controlador invoca dinamicamente uma chamada de sistema (`SIGUSR1` via Docker) para ativar a cNF. Se o tráfego normalizar por 3 ciclos de *polling* consecutivos (*cooldown*), a cNF é desativada (`SIGUSR2`).

C. Resiliência e Auto-Recuperação (Self-Healing)

A rede foi programada para suportar falhas críticas e manter a conformidade do *Slicing*. Esta funcionalidade é gerida por dois módulos em simbiose:

- O *plugin* `table_surveyor.py` audita o estado das tabelas de todos os dispositivos a cada 10 segundos. Para otimizar a comparação, converte as regras P4 instaladas em conjuntos imutáveis (*frozensets*) baseados nos *match fields*. Caso detete deltas negativos (regras apagadas), emite imediatamente um evento do tipo `ENTRY_REMOVED`.
- O *plugin* `slicing_app.py`, que instalou as políticas iniciais através do `slicing.yaml`, mantém subscrições

ativas para estes alertas. Perante um evento de quebra (`ENTRY_REMOVED` ou `DEVICE_RECONNECTED`), executa a recuperação autónoma: força a injeção do *pipeline* P4 (`push_pipeline`) e reinstala as regras de *routing* e limites, restabelecendo a operação em menos de 5 segundos sem intervenção do operador.

D. Comutação L2 Autónoma (Self-Learning)

Para libertar o plano L3 (d2) do processamento intensivo local, a comutação intra-subrede nos dispositivos de borda (d1 e d3) é gerida pelo `self_learning.py`. O *pipeline* L2 utiliza uma sessão de clonagem (*clone session*) acionada para destinos desconhecidos, enviando uma cópia do pacote (marcado com o *EtherType* de sinalização `0x1234`) diretamente para a porta de CPU (510). O *plugin* no controlador interceta este pacote em `PACKET_IN`, extrai o endereço MAC de origem e a porta de ingresso e programa imediatamente as tabelas de *unicast* (`sMacLookup` e `dMacLookup`) com correspondências exatas.

VI. TESTES E RESULTADOS

Para validar a correção da implementação e o cumprimento dos requisitos operacionais definidos no cenário, foram executados múltiplos cenários de teste focados no isolamento, tratamento diferenciado, segurança e reconfiguração dinâmica.

A. Validação do Plano de Controlo (ARP e DHCP)

O primeiro teste validou a correta segregação dos protocolos de gestão para as cNFs de Sinalização. Verificou-se que os *hosts* da LAN 2 (h4, h5, h6) conseguiram obter endereçamento dinâmico com sucesso. O *switch* reencaminhou os pedidos para o DHCP Server, que atribuiu corretamente os *leases* (Figura 1). Em simultâneo, o ARP Proxy demonstrou a capacidade de resolver endereços L3 para L2 de forma isolada (Figura 2).

```

$ ip netns exec veth-dhcp-server listening on veth-dhcp
DHCP Offer: 10.0.2.6 -> aa:00:00:00:02:06
WARNING: No route found for IPv4 destination 255.255.255.255 (no default route?)
DHCP Ack: 10.0.2.6 -> aa:00:00:00:02:06
WARNING: No route found for IPv4 destination 255.255.255.255 (no default route?)
DHCP Offer: 10.0.2.4 -> aa:00:00:00:02:04
WARNING: more No route found for IPv4 destination 255.255.255.255 (no default route?)
DHCP Ack: 10.0.2.4 -> aa:00:00:00:02:04
DHCP Offer: 10.0.2.5 -> aa:00:00:00:02:05
DHCP Ack: 10.0.2.5 -> aa:00:00:00:02:05

```

Fig. 1: Atribuição dinâmica de endereços pela cNF de DHCP

```

$ ip netns exec veth-arp ARP proxy listening on veth-arp
Serving: {'10.0.1.254': 'cc:00:00:00:02:01', '10.0.2.254': 'cc:00:00:00:02:02'}
ARP reply: 10.0.2.254 is at cc:00:00:00:02:02 -> 10.0.2.4
ARP reply: 10.0.2.254 is at cc:00:00:00:02:02 -> 10.0.2.4
ARP reply: 10.0.2.254 is at cc:00:00:00:02:02 -> 10.0.2.4
ARP reply: 10.0.2.254 is at cc:00:00:00:02:02 -> 10.0.2.4
ARP reply: 10.0.2.254 is at cc:00:00:00:02:02 -> 10.0.2.4

```

Fig. 2: Resolução de endereços MAC pela cNF de ARP Proxy

B. Validação da Segurança (Stateful Firewall)

Para testar a topologia assimétrica de segurança, efetuou-se um pedido ICMP (*ping*) a partir do h4 (representando a rede externa/WAN) com destino ao h3 (rede interna/LAN). O *switch* P4, recorrendo à validação rigorosa de cabeçalhos e aos *Bloom Filters*, detetou que o tráfego não tinha sido iniciado internamente e descartou-o de forma proativa. Como

documentado na Figura 3, o resultado foi uma perda de pacotes de 100%, validando a eficácia do bloqueio.

```
root@netsim-vm: /home/netsim/Desktop/PRDS# ping -c 10 10.0.1.3
PING 10.0.1.3 (10.0.1.3) 56(84) bytes of data:
--- 10.0.1.3 ping statistics ---
10 packets transmitted, 0 received, 100% packet loss, time 9200ms
```

Fig. 3: Bloqueio de pedidos ICMP iniciados no exterior pela Firewall P4

C. Telemetria e Isolamento de Slices

Para validar a integridade do isolamento e a precisão da monitorização da infraestrutura, executou-se um teste prático de injeção controlada de tráfego. Utilizou-se um *script* Python interativo no terminal do nó de origem (h2, pertencente à slice Silver) para gerar e enviar uma rajada (*burst*) de exatamente 30 datagramas UDP construídos manualmente com destino ao h4. Como documentado na Figura 4, a rede L3 processou e entregou os dados na íntegra ao destino.

```
root@h2: ~# python3 /home/h2/Desktop/PRDS/traffic_injection.py
[+] Sending 30 UDP packets to 10.0.1.3
[+] All packets received successfully
```

Fig. 4: Injeção de tráfego em rajada (30 pacotes) via script Python no terminal

A eficácia do encaminhamento isolado e da contagem no plano de dados é comprovada pela inteligência do controlador. Através da monitorização contínua (Figura 5), o *plugin* `slicing_counting` avalia a telemetria em tempo real e regista com precisão a receção e aprovação de exatamente 30 pacotes (Total: 30 | Passed(5s): 30) na slice Silver (ID 2). Este registo atesta o sucesso da classificação das slices e demonstra em simultâneo os eventos proativos de *self-learning* processados de forma assíncrona.

```
[controller] d3 rule installed: MyIngress-DMacLearn
[self learning] packet in | deviceid portid src maccc:00:00:00:02:02
[slicing counting] SLICING: Silver (ID 2) | Total: 0 | Passed(5s): 30 | Drops(5s): 0 | BW In: 0.00 Mbps | BW Out: 0.00 Mbps
[slicing counting] SLICING: Gold (ID 3) | Total: 0 | Passed(5s): 0 | Drops(5s): 0 | BW In: 0.00 Mbps | BW Out: 0.00 Mbps
[table:surveyor] ENTRY ADDED on d3
[table:surveyor] ENTRY ADDED on d3
[table:surveyor] ENTRY ADDED on d3
```

Fig. 5: Monitorização contínua: identificação dos 30 pacotes na slice Silver

D. Traffic Shaping Dinâmico

Para testar o *Network Slicing* em condições extremas e validar o encadeamento dinâmico de serviços (*Service Chaining*), gerou-se tráfego UDP intensivo utilizando a ferramenta *iperf*. O teste consistiu em injetar tráfego superior ao limite contratualizado (ex: 15 Mbps para a slice Silver). Como se pode observar na Figura 6, o débito recebido no destino (Node: h4) é estrangulado pelo sistema, estabilizando sem ultrapassar o *rate* imposto pelo SLA.

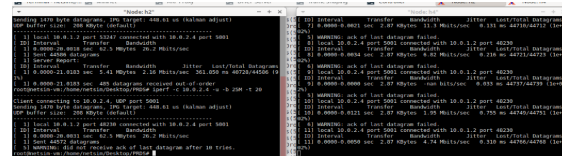


Fig. 6: Teste iperf demonstrando o limite de largura de banda aplicado rigorosamente à slice

Este isolamento é garantido pela intervenção autónoma do controlador, cumprindo o requisito de ausência de intervenção manual. Ao detetar, através da telemetria, pacotes marcados como *drops* nas tabelas P4, o *plugin* envia imediatamente um sinal de sistema (SIGUSR1) à cNF de retenção, ativando-a (Figura 7).

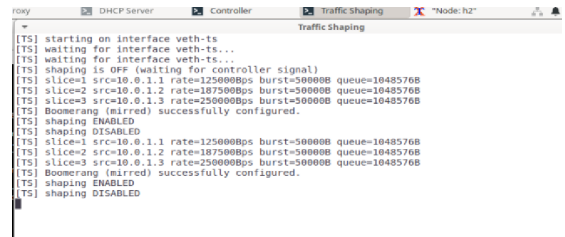


Fig. 7: Interface da cNF de Traffic Shaping a receber o sinal de ativação autónoma do controlador

A cNF passa então a armazenar ativamente o tráfego desviado nas suas filas HTB, introduzindo *delay* e alisando a saída (*shaping*) em vez de o descartar de forma abrupta na *hardware*. A Figura 8 ilustra os registos contínuos de telemetria do controlador, que reportam com exatidão a ativação do estado (`traffic-shaping ENABLED`) bem como as métricas relativas aos débitos de entrada e saída por cada uma das três slices, confirmando a robustez do *Extended Data Plane*.

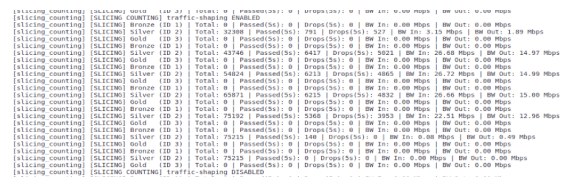


Fig. 8: Telemetria do controlador confirmando a ativação autónoma e a medição de débitos (In/Out)

Para além disso, a telemetria permite observar o comportamento intrínseco do mecanismo de retenção de tráfego no exato momento em que a injeção do *iperf* termina. Como evidenciado na Figura 8, existe um instante final na slice Silver em que o *throughput* de entrada (BW In) desce drasticamente para um valor residual (0.08 Mbps), mas a taxa de saída (BW Out) mantém-se num valor superior (0.49 Mbps). Este diferencial temporal prova que os pacotes em excesso não foram descartados no pico de tráfego. Em vez disso, foram armazenados em memória nas filas HTB da cNF e libertados de forma contínua e controlada para a rede até ao esvaziamento total do *buffer*.

E. Resiliência da Rede (Self-Healing)

O teste mais crítico incidu sobre a fiabilidade e autonomia do plano de controlo perante anomalias graves na infraestrutura. Para o efeito, desenhou-se um cenário de falha induzida em tempo de execução.

Primeiramente, estabeleceu-se uma transmissão contínua de dados (*iperf*) do h3 para o h4, criando um fluxo de tráfego estável e validando que o encaminhamento L3 operava com normalidade (Figura 9).

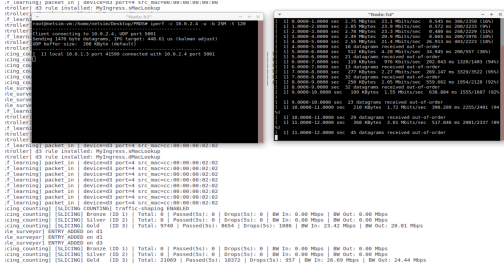


Fig. 9: Tráfego inicial estável antes da indução da falha

Durante esta transmissão, foi simulada uma falha crítica, em que foram limpas manualmente a tabela de encaminhamento principal (`table_clear MyIngress.ipv4Lpm`) através da CLI de gestão do *switch* d2 (Figura 10). Esta ação interrompeu imediatamente o fluxo de dados, simulando a perda de estado no *Data Plane*.

```
=====
TABLE ENTRIES
=====
Dumping entry 0x0
Match key:
* ipv4.dstAddr      : LPM      0a000101/32
Action entry: MyIngress.forward - 01, aa0000000101
=====
Dumping entry 0x1
Match key:
* ipv4.dstAddr      : LPM      0a000102/32
Action entry: MyIngress.forward - 01, aa0000000102
=====
Dumping entry 0x2
Match key:
* ipv4.dstAddr      : LPM      0a000103/32
Action entry: MyIngress.forward - 01, aa0000000103
=====
Dumping entry 0x3
Match key:
* ipv4.dstAddr      : LPM      0a000204/32
Action entry: MyIngress.forward - 02, aa0000000204
=====
Dumping entry 0x4
Match key:
* ipv4.dstAddr      : LPM      0a000205/32
Action entry: MyIngress.forward - 02, aa0000000205
=====
Dumping entry 0x5
Match key:
* ipv4.dstAddr      : LPM      0a000206/32
Action entry: MyIngress.forward - 02, aa0000000206
=====
Dumping default entry
Action entry: MyIngress.drop -
=====
RuntimeCmd: table_clear MyIngress.ipv4Lpm
RuntimeCmd: table_clear MyIngress.ipv4Lpm
RuntimeCmd: table_dump MyIngress.ipv4Lpm
=====
TABLE ENTRIES
=====
Dumping default entry
Action entry: MyIngress.drop -
=====
RuntimeCmd: █
```

Fig. 10: Simulação de falha: Limpeza manual da tabela de roteamento via CLI

Apenas decorridos alguns segundos, o módulo `table_surveyor` detetou o *delta* negativo através

da sua auditoria periódica e propagou o evento crítico `ENTRY_REMOVED`. O módulo orquestrador (`slicing_app`) reagiu de imediato à anomalia, alertando no terminal a necessidade de forçar a auto-recuperação da rede (*"Rule removal detected... Forcing auto-recovery"*), conforme ilustrado na Figura 11.

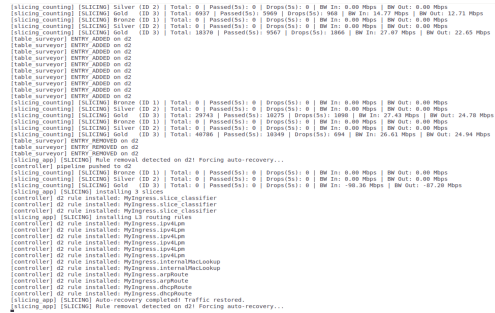


Fig. 11: Controlador a detetar as regras em falta e a iniciar o processo de *Self-Healing*

Finalmente, o controlador executou a injeção forçada do *pipeline* (`push_pipeline`) e reinstalou autonomamente todo o estado da rede, incluindo as rotas L3, ARP, DHCP e de Slicing. A Figura 12 documenta a conclusão bem-sucedida do processo (*"Recovery completed successfully"*), momento após o qual a comunicação aplicacional foi rapidamente restabelecida sem qualquer intervenção humana, validando a resiliência da arquitetura implementada.

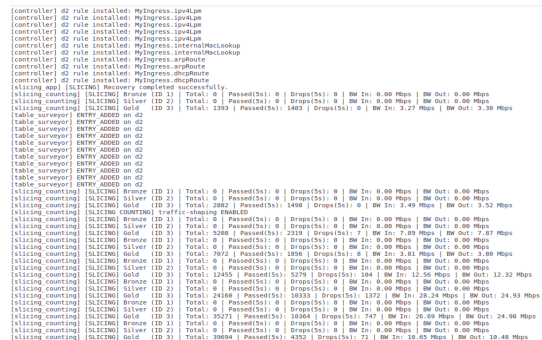


Fig. 12: Confirmação da reinstalação das regras e sucesso da recuperação

VII. CONCLUSÃO

O presente trabalho demonstrou com sucesso a conceção e implementação de uma arquitetura de Network Slicing dinâmica e resiliente, utilizando o paradigma Software-Defined Networking (SDN) em conjunto com a linguagem P4 para o processamento de pacotes à velocidade da linha.

Durante o desenvolvimento, o maior desafio arquitetural encontrado foi a orquestração do Traffic Shaping fora do switch L3. Implementar um encaminhamento do tipo "bumerangue", onde o pacote sai do switch para a porta virtual do container (veth) e, após o delay imposto pelo protocolo HTB, volta a reentrar no switch pela mesma porta, gerou problemas

complexos de dupla contagem (double-counting) nos medidores e contadores. Foi necessário refinar meticulosamente a lógica do pipeline de entrada (MyIngress) para garantir que o pacote de regresso fosse devidamente identificado e transitasse livremente sem reiniciar os contadores de tráfego original, evitando assim ciclos infinitos (loops) e métricas adulteradas.

Outro desafio importante prendeu-se com o mapeamento eficiente de estado no Control Plane para garantir a resiliência da rede através da auditoria de tabelas. Implementar comparações de polling exaustivas entre potenciais milhares de regras gRPC em bruto mostrou-se ineficiente. Uma vez que os objetos devolvidos pelo P4Runtime contêm múltiplos metadados e estruturas aninhadas, iterar sobre eles com ciclos tradicionais a cada 10 segundos (para todos os switches da topologia) criaria um estrangulamento nos recursos do CPU do controlador. A solução arquitetada contornou este problema extraindo apenas a essência de cada regra, os seus campos de correspondência (match fields), e convertendo essas propriedades imutáveis em frozensets em Python. Ao transformar as regras nestes conjuntos matemáticos estáticos e "hasháveis", o sistema passou a poder utilizar operações nativas e altamente otimizadas de subtração de conjuntos (calculando chaves anteriores vs chaves atuais). Esta abordagem computacionalmente muito mais leve permite calcular as diferenças (deltas) quase instantaneamente, revelando-se a essencial para a agilidade de deteção do mecanismo de Self-Healing.

O projeto consolidou a compreensão empírica sobre a divisão estrita das funções de rede (Function Placement), demonstrando que o hardware e o software resolvem problemas distintos na arquitetura. O hardware (P4) atua como a via rápida da rede (Fast-Path), onde o processamento em hardware dedicado provou ser imbatível em lógicas binárias de classificação rápida, como o bloqueio de pacotes via Firewall e a marcação de QoS, operando sem latência perceptível. No entanto, devido à ausência de memória dinâmica extensível, o P4 é incapaz de criar buffers ou gerir temporizadores complexos. Por seu lado, o software (cNFs) atua como o ecossistema ideal para compensar estas limitações, uma vez que os containers Linux oferecem a memória RAM e a flexibilidade de CPU necessárias para a gestão de estado temporal. Estas características revelaram-se imperativas para reter tráfego em filas hierárquicas para Traffic Shaping e para manter os diálogos assíncronos e retenções prolongadas de protocolos de sinalização como o DHCP e o ARP. Em suma, delegar a gestão de estado complexo para as cNFs isoladas garante a estabilidade do sistema, libertando o hardware para a sua vocação principal: a comutação de tráfego à velocidade do link.

Em suma, a solução entregue atinge o equilíbrio pretendido entre altíssimo desempenho de forwarding e flexibilidade de software. Cumpre rigorosamente os requisitos de isolamento das slicing lógicas e apresenta uma infraestrutura proativa, dotada de telemetria inteligente e capacidade autónoma de adaptação e auto-recuperação perante falhas operacionais críticas.